PROYECTO FINAL

Cuna inteligente con ESP32, MQTT y panel web

Preparado por:
Luisdavid Colina V24766381
Samantha Ramírez V31307714

1. Planteamiento

Quien cuida a un bebé no puede estar siempre junto a la cuna. Durante la noche hay tres situaciones que piden atención inmediata: el llanto, el pañal mojado y un cuarto demasiado caliente o demasiado frío. Una cuna común no avisa de ninguna de las tres.

El proyecto propone una cuna que detecta esas situaciones, responde por sí sola con un movimiento suave y avisa a los padres desde el teléfono. El trabajo se realizó completo en simulación sobre Wokwi, con una placa ESP32 y un broker MQTT público.

Los objetivos son los siguientes.

1. Detectar el llanto y mecer la cuna con una intensidad que crece si el llanto continúa.
2. Medir la temperatura y la humedad del cuarto y controlar un ventilador.
3. Avisar al teléfono cuando el pañal se moja, hay gas o humo, o el llanto se prolonga.
4. Permitir mecer la cuna y manejar el ventilador a distancia desde un panel web.

2. Antecedentes

Las cunas inteligentes son un tema frecuente en proyectos de IoT. Una lista de proyectos para estudiantes de ingeniería incluye la cuna que mece al detectar el llanto y vigila la temperatura del cuarto [1]. Un proyecto publicado en Intel DevMesh usa un ESP32 con un sensor de sonido, un DHT11 y un motor que inclina la cuna, con alertas por la aplicación Blynk [2]. Un proyecto de grado de la Universidad Nacional An-Najah añade un sensor de humedad para el pañal, un ventilador que se enciende cuando el cuarto supera los 30 °C y el control remoto del mecido desde una aplicación [3].

Un artículo de 2025 describe un sistema más completo. Reporta niveles de balanceo del 1 al 5, parada automática del balanceo a los cinco minutos, un rango seguro de temperatura de 20 a 25 °C con enfriamiento por encima y calefacción por debajo, y un panel con historial y control remoto [4]. El mismo artículo presenta cifras de exactitud que no son consistentes entre sus secciones, por lo que este informe toma de él solo los parámetros de diseño y no sus resultados.

Un artículo de 2026 sobre una cuna con ESP32 y Blynk describe que el controlador separa los ruidos breves del llanto verdadero para no mecer con cada sonido, y que la cuna empieza a moverse en menos de 2,5 segundos tras confirmar el llanto [5]. De ahí sale la condición de duración mínima que usa este proyecto.

La cuna de este informe toma esos parámetros y añade cuatro elementos: un escalonamiento de la respuesta según cuánto dura el llanto, un control con histéresis para el ventilador, un panel que se maneja por MQTT con las mismas órdenes que entiende el programa y un aviso al teléfono que no depende de tener el panel abierto.

3. Diseño del sistema

3.1. Arquitectura

El ESP32 lee los sensores y maneja los actuadores. Publica su estado en un broker MQTT al instante cuando algo cambia, y además envía un latido cada segundo. El panel web se suscribe a ese estado para mostrarlo. Las órdenes hacen el camino inverso: el panel publica en un tópico de comandos y el ESP32 las recibe. Los avisos urgentes salen del ESP32 por HTTP hacia ntfy, que los entrega al teléfono aunque el panel esté cerrado. La figura 1 resume el flujo.

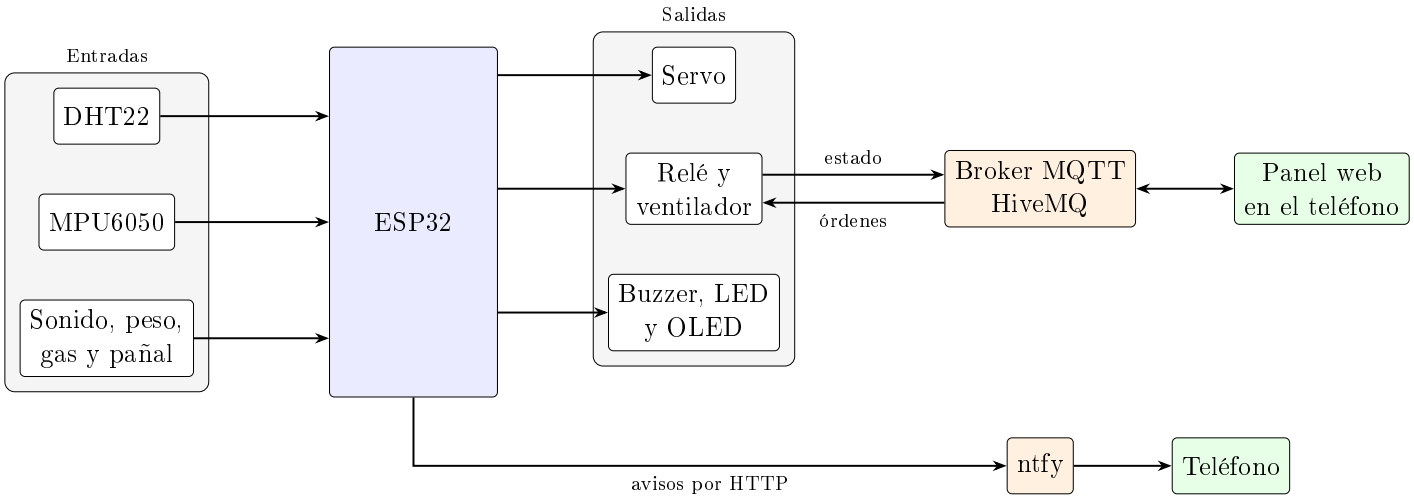


Figura 1: Arquitectura del sistema. El estado sube al cambiar y con un latido por segundo, las órdenes bajan por un tópico distinto y los avisos urgentes van al teléfono.

3.2. Componentes y conexiones

La tabla siguiente lista cada pieza del circuito, la magnitud que mide o la acción que realiza, y el pin del ESP32 al que se conecta.

Pieza	Función	Pin
DHT22	Temperatura y humedad del cuarto	15
MPU6050	Movimiento de la cuna, por I2C	21 y 22
Pantalla OLED SSD1306	Estado de la cuna, por I2C	21 y 22
Deslizante Sonido	Nivel de ruido, de 0 a 4095	34
Deslizante Peso	Peso sobre el colchón, de 0 a 8 kg	33
Deslizante Gas	Concentración de gas o humo	35
Deslizante Pañal	Humedad del pañal	32
Servo	Mece la cuna	18
Módulo relé	Enciende el ventilador	26
LED rojo	Alerta de llanto, pañal o gas	27
Buzzer	Avisos sonoros	25
Cuatro botones	Provocan situaciones de prueba	13, 14, 16 y 17

El dibujo del circuito sigue dos reglas: cada pieza va del lado del ESP32 donde está su pin, en el mismo orden, y la alimentación usa símbolos VCC y GND propios de cada pieza en vez de llevar todos los cables a un solo pin. Así ningún cable cruza la placa, como muestra la figura 2. Los símbolos VCC del simulador entregan 3,3 V: cada deslizante recorre de 0 a 4095 en el convertidor sin saturarse.

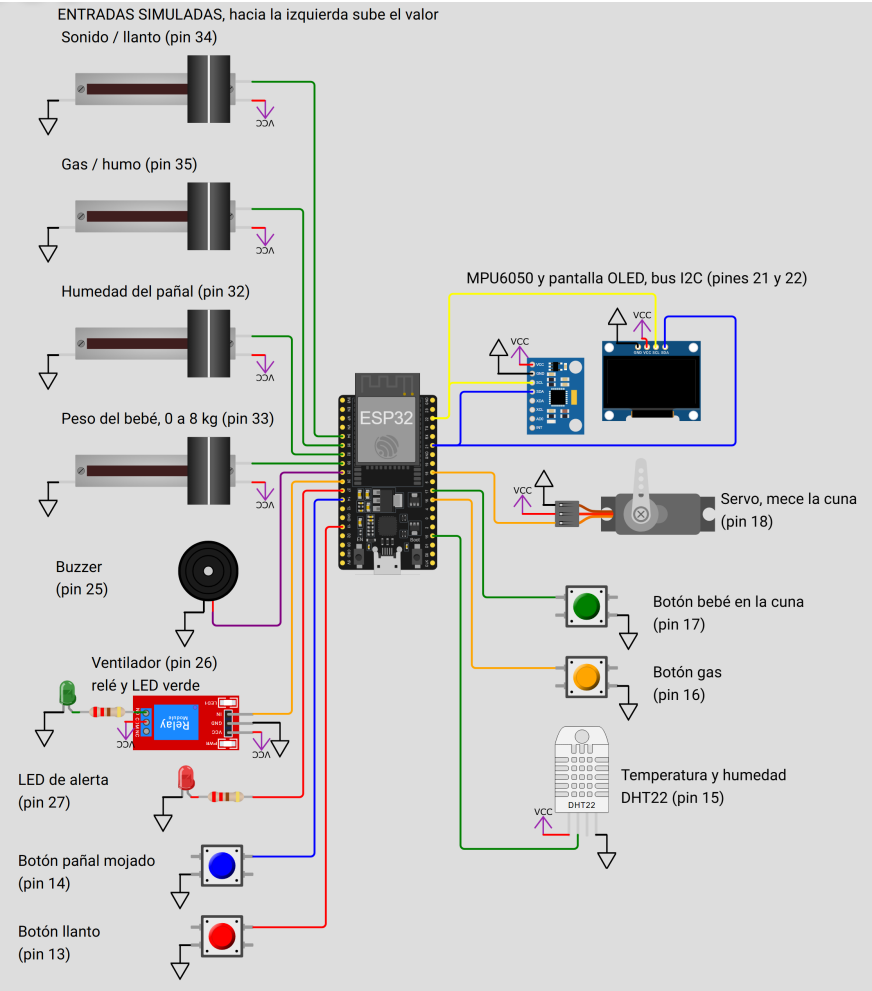


Figura 2: Circuito en Wokwi. Los deslizantes están girados para que su señal quede frente al ESP32, por lo que su valor sube hacia la izquierda.

3.3. Sustituciones por las limitaciones del simulador

La documentación de Wokwi no incluye un sensor de sonido entre sus piezas. Incluye el amplificador de celdas de carga HX711, pero su valor solo se controla desde escenarios de automatización y no con un control que se mueva a mano durante la demostración [7]. El sensor de gas MQ2 sí existe, pero solo permite fijar la concentración inicial [8].

Por eso las cuatro entradas que representan sonido, peso, gas y humedad del pañal son controles deslizantes que entregan un voltaje analógico. El programa las lee con el convertidor del ESP32 y las trata como lo haría con los sensores reales. En un montaje físico, el deslizante de sonido sería un micrófono con salida analógica, el de peso una celda de carga con HX711, el de gas un MQ2 y el del pañal un sensor de humedad. El resto de las piezas, como el DHT22, el MPU6050, el servo, el relé y la pantalla OLED, sí existen en el simulador y se usan tal cual.

4. Lógica de control

4.1. Respuesta al llanto

El programa lee el sensor de sonido cada 200 ms, tomando el valor máximo de veinte muestras para no perder los picos. Un ruido alto aislado no cuenta como llanto. Debe mantenerse por encima del umbral durante dos segundos y, además, debe haber un bebé en la cuna según la lectura de peso. Cumplidas las dos condiciones, la cuna empieza a mecer con intensidad suave.

Si el llanto continúa, la respuesta escala. Al minuto el mecido pasa a intensidad firme. A los tres minutos se avisa a los padres de un llanto prolongado, con una alarma sonora, un mensaje al panel y una notificación al teléfono. Cuando el sonido baja del umbral y se mantiene así durante veinte segundos, se da al bebé por calmado y el mecido se detiene. La figura 3 muestra la secuencia.

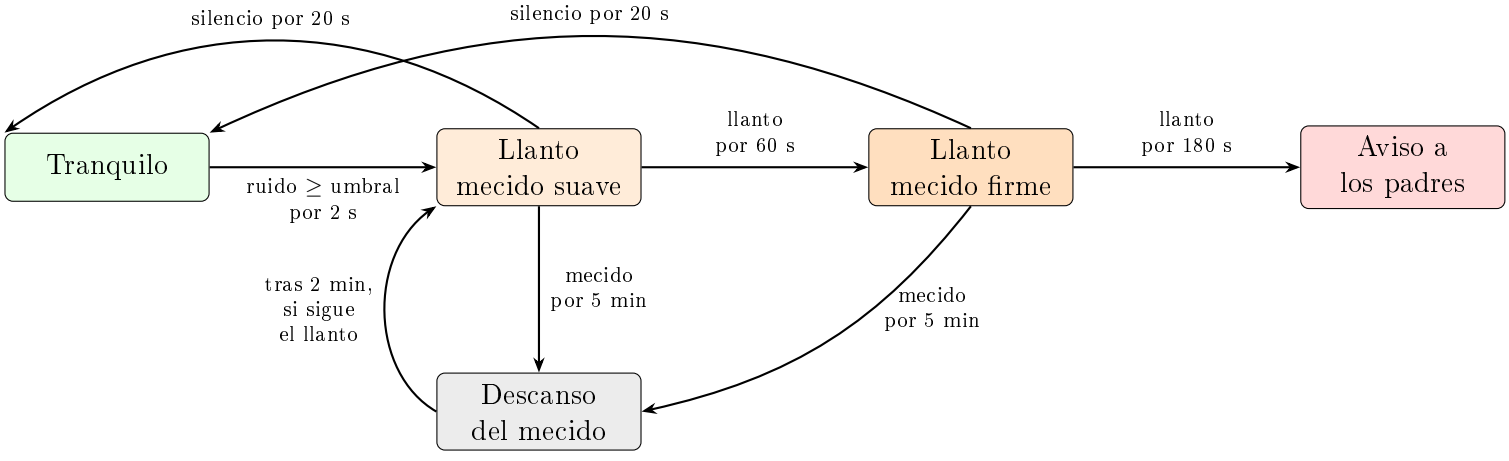


Figura 3: Estados de la respuesta al llanto. El mecido se detiene siempre a los cinco minutos y descansa dos minutos antes de poder reiniciarse solo. El aviso a los padres no detiene el mecido.

El límite de cinco minutos aplica también cuando el balanceo se inicia a mano desde el panel. Si el bebé sale de la cuna, el programa da el llanto por terminado y detiene el mecido automático.

Esos tiempos no caben en una exposición, y menos en un simulador que corre más lento que el tiempo

real. Por eso existe un modo demo: se activa desde el panel, sin recompilar, y divide todos los tiempos entre diez. Los valores de este informe son los del modo normal.

4.2. Ambiente del cuarto

El ventilador se enciende cuando la temperatura llega a 26 °C y se apaga al bajar a 24,5 °C. La diferencia de un grado y medio entre los dos valores evita que el relé se active y se desactive sin parar cuando la temperatura oscila alrededor de un solo umbral. El panel permite además forzar el ventilador encendido o apagado.

Se envía un aviso cuando la temperatura llega a 28 °C o baja a 20 °C, y otro cuando la humedad cae por debajo de 40 %. Cada aviso se emite una sola vez y se rearma cuando la medida vuelve a un margen seguro, de modo que el panel no se llena de mensajes repetidos.

El gas y el pañal también tienen rearme. La alarma de gas se activa con una lectura de 1800 y se apaga por debajo de 1400, y la del pañal se activa con 2000 y se apaga por debajo de 1500. Al apagarse se publica un aviso de gas despejado o de pañal seco. El MPU6050 vigila los golpes: una desviación de más de 0,5 g respecto a la gravedad genera un aviso de movimiento brusco, que se rearma por debajo de 0,2 g.

Situación	Condición	Respuesta
Llanto	Sonido alto por 2 s con bebé presente	Mecido suave
Llanto sostenido	60 s de llanto	Mecido firme
Llanto prolongado	180 s de llanto	Alarma y aviso
Calma	20 s bajo el umbral	Detiene el mecido
Cuarto caliente	26 °C o más	Ventilador encendido
Cuarto fresco	24,5 °C o menos	Ventilador apagado
Temperatura fuera de rango	28 °C o más, 20 °C o menos	Aviso
Humedad baja	Menos de 40 %	Aviso
Pañal mojado	Lectura sobre el umbral	Aviso y pitido
Gas o humo	Lectura sobre el umbral	Aviso y alarma continua
Movimiento brusco	Más de 0,5 g de desviación	Aviso y LED

5. Comunicación por MQTT

El ESP32 se conecta a la red virtual `Wokwi-GUEST`, que no pide contraseña y tiene salida a internet [6]. Desde ahí publica en el broker público de HiveMQ mediante la librería PubSubClient. Todos los tópicos cuelgan de `ucv6565/cuna/` seguido de un identificador propio, para que los mensajes de este proyecto no se mezclen con los de otros que usen el mismo broker.

Tópico	Dirección	Contenido
`.../state`	Cuna al panel	Estado en JSON al cambiar y cada segundo, retenido
`.../event`	Cuna al panel	Texto de cada aviso
`.../status`	Cuna al panel	`online` u `offline`, retenido
`.../cmd`	Panel a la cuna	Orden en texto, como `rock:2`

El estado se publica con la bandera de retención, de modo que un panel que se abre después recibe de inmediato la última lectura. Ese dato puede ser viejo, así que el panel lo muestra como último dato conocido y solo declara la cuna en línea cuando llega un estado en vivo. El tópico `.../status` usa el

último deseo de MQTT (LWT): al conectarse la cuna publica `online` y le deja al broker el mensaje `offline`, que el broker publica por ella si deja de responder.

Las órdenes que entiende la cuna son `rock:0` a `rock:3` para detener o mecer, `fan:auto`, `fan:on` y `fan:off` para el ventilador, `cry:` y `temp:` seguidos de un valor para cambiar los umbrales, `beep` para probar el buzzer y `demo:on` o `demo:off` para el modo demo. Las de simulación, como `sim:cry:1` y `sim:cry:0`, fijan un valor en vez de alternarlo, y `sim:temp:27` fuerza una temperatura. Repetir cualquiera de ellas no cambia nada, lo que permite al panel reintentarlas sin efectos secundarios.

La reconexión de WiFi y MQTT corre en una tarea aparte de FreeRTOS. Mientras dura, el programa principal sigue leyendo los sensores, meciendo la cuna y haciendo sonar las alarmas.

El broker público no tiene autenticación, así que cualquiera que conozca el identificador podría leer el estado o enviar órdenes. Para una simulación académica es aceptable. Un producto real necesitaría un broker propio con usuario, contraseña y cifrado.

6. Avisos al teléfono

Los avisos de gas, llanto prolongado, pañal mojado y movimiento brusco salen del ESP32 por HTTP hacia el servicio ntfy [9], en el tema `ucv6565-cuna-` seguido del identificador. Quien instala la aplicación y se suscribe a ese tema los recibe aunque el panel esté cerrado. Se eligió sobre un bot de Telegram porque no necesita token, que quedaría a la vista en el código público del simulador. A cambio, cualquiera que conozca el nombre del tema puede leer los avisos.

El envío lo hace la misma tarea de red, a partir de una cola, para no detener el programa principal. Cada tipo de aviso tiene un mínimo de 30 s entre envíos.

7. Panel web

El panel es una sola página HTML que se abre desde el teléfono o el computador. Se conecta al mismo broker por WebSocket seguro y se suscribe a los tópicos de estado, de avisos y de presencia. Separa el estado del broker del estado de la cuna, indica hace cuánto llegó el último dato y estima a qué velocidad corre la simulación a partir del tiempo de encendido del ESP32. Con la cuna fuera de línea atenúa los datos y desactiva los controles.

Muestra el estado general de la cuna, una escena animada, las medidas de temperatura, humedad, peso, sonido, gas, pañal y movimiento, el ángulo del servo, y el estado del LED y del buzzer. Las alarmas de pañal y gas las decide el firmware: el panel las lee del estado y no recalcula los umbrales, para no contradecir la histéresis.

Cada botón pasa por tres estados: pendiente, confirmado cuando el estado de la cuna refleja la orden, y “la cuna no respondió” si tras tres intentos no hay confirmación. Además de mecer y manejar el ventilador, permite cambiar los umbrales, probar el buzzer, forzar la temperatura y activar el modo demo.

Hay una gráfica por cada medida, sobre el mismo eje de cinco minutos, con los huecos visibles cuando faltan datos y una tabla con los mismos valores. Gráficas y avisos se conservan al recargar. Un botón

activa la notificación del navegador y un sonido para los eventos críticos, solo mientras la página siga abierta.

La librería MQTT del panel va incluida en la carpeta del proyecto, de modo que la página solo necesita conexión para llegar al broker.

8. Resultados de la simulación

Las pruebas se ejecutaron el 1 de noviembre de 2026 con el programa y el circuito finales, en copias temporales del proyecto de Wokwi con un identificador propio. Un script con Playwright manejó Wokwi y el panel publicado en Vercel desde un navegador sin interfaz, y un cliente MQTT tomó los tiempos. Se agruparon en cinco escenarios. Cada versión se midió en una sola corrida, por lo que los valores de tiempo cambian según el equipo. Los escenarios con Wokwi CLI de la carpeta `pruebas/` no se repitieron con esta versión porque requieren un token.

Escenario 1: tiempo real

La tabla compara el programa anterior con el final. La causa de la lentitud no era el broker, que respondió en una mediana de 234 ms ida y vuelta, sino la simulación: sin ninguna pausa en `loop()` y con el estado saliendo cada 2 s del reloj simulado, un dato podía tardar más de 20 s reales. Las mejoras fueron una pausa de 10 ms en cada ciclo, publicar el estado al cambiar, dibujar la pantalla solo al cambiar, I2C a 400 kHz y conectar el WiFi indicando el canal.

Medida	**Antes**	**Después**
Velocidad en Wokwi, mediana	8 %	25 %
Orden hasta el estado, mediana	4 de 5 sin llegar en 20 s	1,10 s
Órdenes sin respuesta	–	0 de 12
Reconexiones al broker	1 en unos 4 min	0 en 6 min
Panel con la simulación detenida	En línea hasta 40 s	Sin señal a los 16 s
Aviso de cuna apagada por LWT	No existía	45 s

En la versión final, el tiempo de la orden al estado fue de 0,83 s como mínimo y 5,90 s como máximo, con dos de las 12 órdenes por encima de 2 s. El primer dato tardó entre 70 y 87 s desde que se pulsa iniciar, sobre todo por la compilación. Al recargar el panel con la simulación detenida, nunca indicó que la cuna estuviera en línea.

Escenario 2: respuesta al llanto con escalada

Se corrió en modo demo, con los tiempos divididos entre diez, y con el bebé presente. El mecido se había detenido a mano justo antes, para comprobar que esa pausa no bloquea la demostración. La tabla muestra el momento de cada aviso desde que sube el sonido. El tiempo simulado se lee del reloj del ESP32, con resolución de un segundo. La figura 4 muestra el panel durante este escenario.

Aviso	Programado	Simulado	Reloj real
Llanto detectado y mecido suave	0,2 s	menos de 1 s	1,2 s
Mecido firme	6 s	6 s	11,9 s
Llanto prolongado y aviso al teléfono	18 s	18 s	32,2 s
Mecido detenido por el límite	30 s	30 s	53,2 s
Bebé calmado, 2 s tras bajar el sonido	–	33 s	57,6 s

Las duraciones sin dividir, de 60 s, 180 s y 5 minutos, no se ejecutaron de principio a fin en el reloj normal. El filtro de ruidos sí se probó con los tiempos normales: tres ruidos de hasta un segundo simulado no dispararon el llanto, y uno sostenido sí.



Figura 4: Panel web en modo demo durante el llanto, con la cuna en línea y el mecido en nivel 1.

Escenario 3: ventilador con histéresis

Se forzó la temperatura a 24, 25, 27, 25 y 24 °C, en ese orden, y se leyó el estado del LED verde en el simulador. A 25 °C, que está entre los dos umbrales, el ventilador siguió apagado al subir y siguió encendido al bajar. Encendió a 27 °C y apagó a 24 °C. Moviendo el control del DHT22 se repitió la bajada de 27 a 25 y 24 °C, con el mismo resultado. La figura 5 muestra el panel con el ventilador activo y el aviso de pañal mojado.

Escenario 4: gas, pañal y movimiento

En cada paso se comparó la alarma del firmware, el LED rojo del simulador y lo que muestra el panel, y coincidieron siempre. La lectura de gas se llevó a 2202, 1601 y 1201: la alarma se activó, siguió activa



Figura 5: Panel web con el ventilador encendido por calor forzado y el pañal mojado.

entre el rearme y el umbral, y se apagó. La del pañal se llevó a 2402, 1801 y 1201, con el mismo comportamiento. Con 2 g en el MPU6050 se generó el aviso de movimiento brusco con el LED rojo, y al volver a 1 g se rearmó. La figura 6 muestra el panel durante la alerta de gas.

Escenario 5: avisos, controles y entradas

Los cuatro tipos de aviso llegaron a ntfy, comprobados consultando el tema (figura 7). Durante un envío, con la alarma de gas sonando, el estado siguió llegando con un máximo de 2,8 s entre mensajes y el servo siguió moviéndose. No se probó la recepción en un teléfono físico.

Se probaron los cuatro botones del circuito y 28 usos de los controles del panel publicado, y todos quedaron confirmados por el estado de la cuna. Los deslizantes de sonido, gas y pañal leyeron 0, 2050 y 4095 al 0, 50 y 100 % de su recorrido, y el de peso 0, 4 y 8 kg.

9. Conclusiones

1. Una cuna que responde por escalones según la duración del llanto ignora los ruidos de menos de dos segundos y reserva el aviso al teléfono para los casos prolongados.

2. El control con histéresis se comprobó en los dos sentidos: entre 24,5 y 26 °C el relé mantiene su estado anterior.

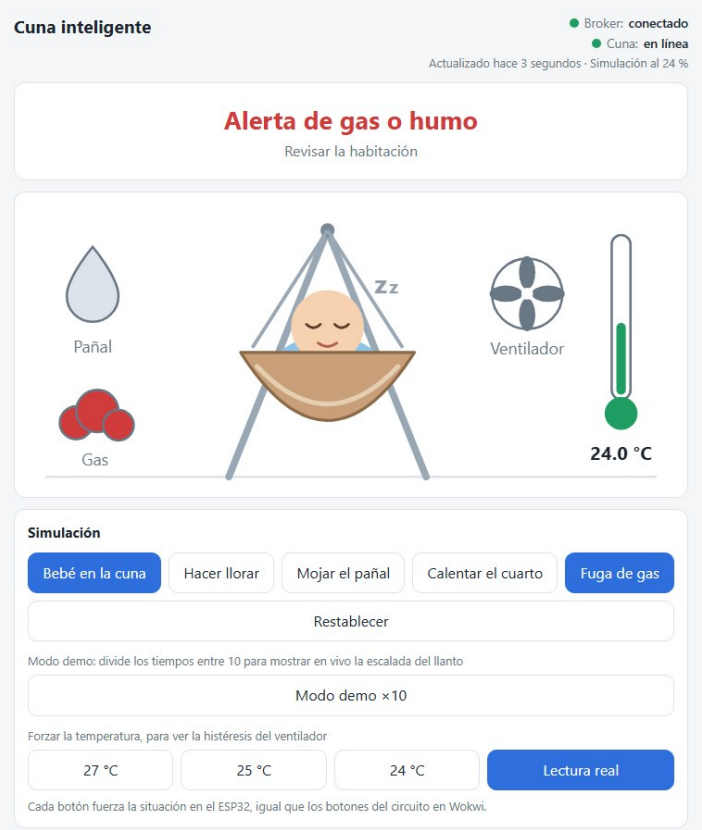


Figura 6: Panel web con la alerta de gas activa.

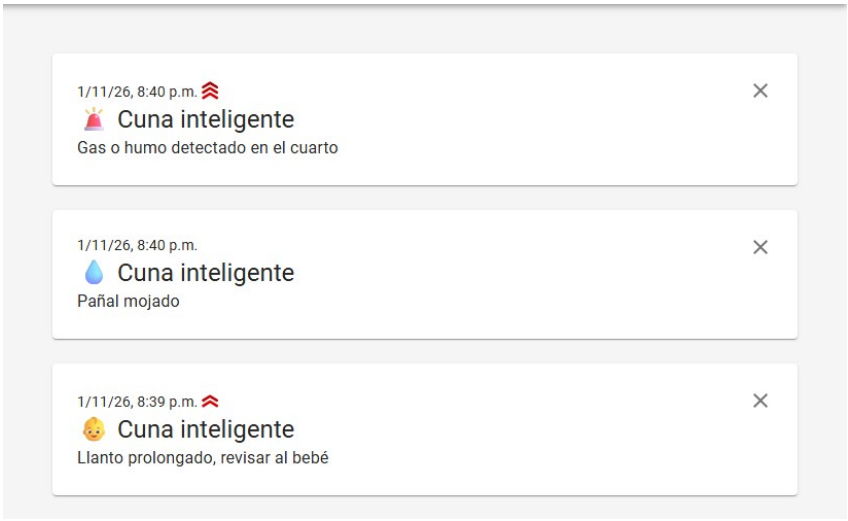


Figura 7: Avisos recibidos en el tema de ntfy, vistos en su página web.

3. El tiempo real en un simulador hay que medirlo. El broker no era el problema: con pausas en el programa y publicando al cambiar, la mediana de la orden al estado bajó de más de 20 s a 1,10 s.

4. MQTT permite separar la cuna del panel. Los dos solo comparten el broker y el formato de los mensajes, y con el último deseo el panel distingue un dato en vivo de uno viejo.

5. Las limitaciones del simulador se resolvieron con controles deslizantes equivalentes a las señales analógicas de los sensores reales, y esa sustitución queda declarada en el informe. Todo se verificó en simulación: los umbrales no están calibrados con sensores reales.

Referencias

[1] Robocraze, "25 Creative IoT Lab Projects for Engineering Students". https://robocraze.com/blogs/post/25-creative-iot-lab-projects-for-engineering-students

[2] Intel DevMesh, "Smart Cradle System". https://devmesh.platformos.com/projects/smart-cradle-system

[3] Universidad Nacional An-Najah, repositorio institucional, proyecto de grado "Comfort Sense baby bed", 2024. https://repository.najah.edu/bitstreams/dd7b1d37-ed68-477b-893e-2b435eaeec99/download

[4] "A novel smart baby cradle system utilizing IoT sensors and machine learning for optimized parental care". PubMed Central, 2025. https://pmc.ncbi.nlm.nih.gov/articles/PMC12125208/

[5] P. S. Pise et al., "IoT-Based Intelligent Baby Cradle", International Journal of Innovative Research in Technology, vol. 12, n.º 12, mayo de 2026. https://ijirt.org/publishedpaper/IJIRT202278_PAPER.pdf

[6] Wokwi Docs, "ESP32 WiFi Networking". https://docs.wokwi.com/guides/esp32-wifi

[7] Wokwi Docs, "wokwi-hx711". https://docs.wokwi.com/parts/wokwi-hx711

[8] Wokwi Docs, "wokwi-gas-sensor". https://docs.wokwi.com/parts/wokwi-gas-sensor

[9] ntfy, "Send push notifications to your phone or desktop via PUT/POST". https://docs.ntfy.sh/